MIU TECHCIRCLE

PRESENTED BY

- SARAH WAEL
- SELVIA IBRAHIM
- HANA MOHAMED
- MOSTAFA YASSER
- ISMAIL AHMED

What is MIU TECHCIRCLE

MIU Tech Circle is a mobile application designed to connect the Computer Science community at MIU in one unified platform.

It brings together students, alumni, doctors, and teaching assistants to bridge the gap between academic learning and real-world experience.

The app provides structured access to opportunities such as internships, projects, competitions, events, and freelancing, while offering clear guidance based on the student’s academic level.

Through organized announcements, secure communication, and active alumni engagement, MIU Tech Circle helps students build career readiness, stay informed, and grow within a safe and trusted academic environment.

The Problem

01

MIU Computer Science students are highly motivated and capable, but lack a direct system that helps them grow professionally.

02

Career opportunities (internships, projects, competitions) are scattered across multiple platforms such as LinkedIn and Wuzzuf.

03

These platforms are not designed for undergraduate students and provide no clear guidance.

04

Students struggle to understand:
Where to start
What skills the market actually needs
Which opportunities are relevant to their level

The Gap

There is no structured connection between academic learning and real-world practice.

The alumni network is underutilized, despite having strong industry experience.

Opportunities and announcements are unorganized, causing many valuable chances to be missed.

As a result:

Students lose real opportunities

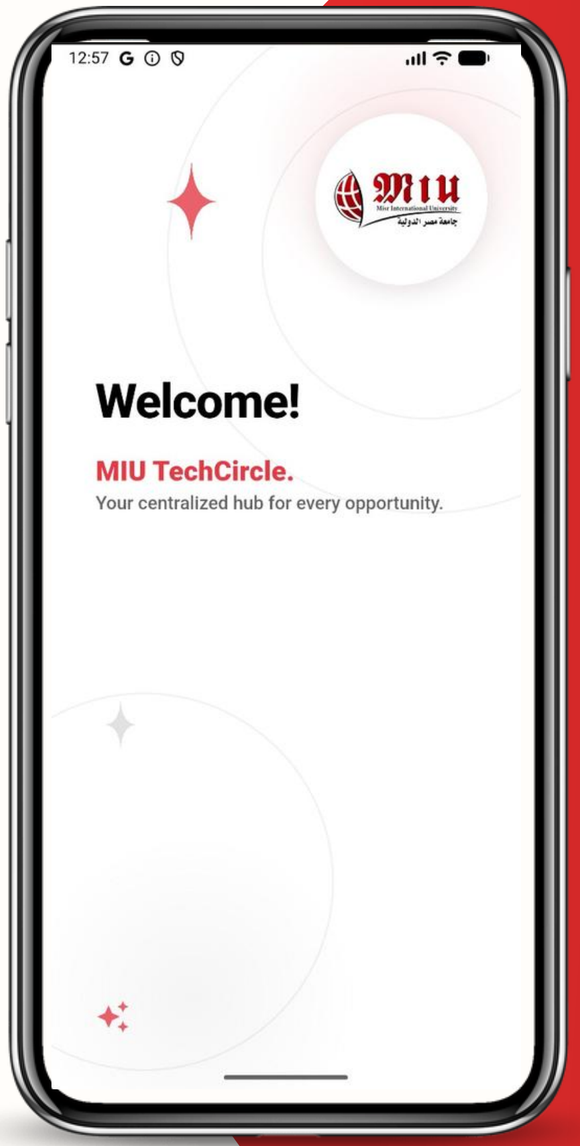
Career paths become unclear

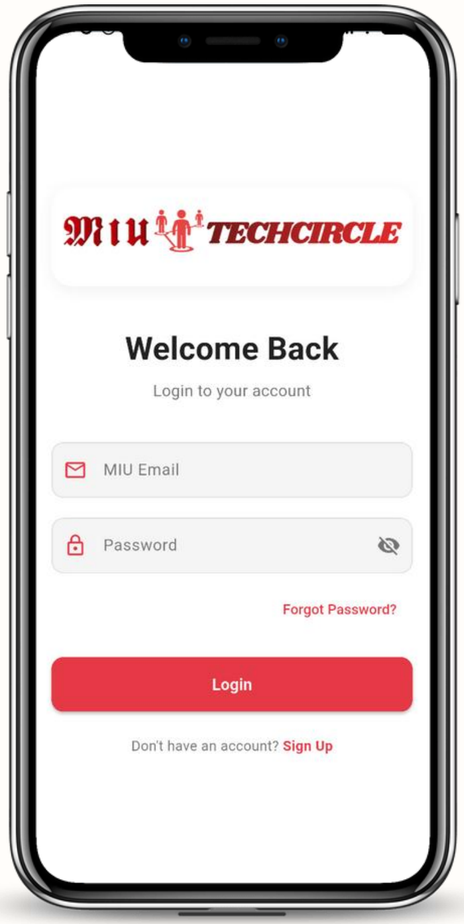
Student potential remains unused

LinkedIn	Facebook Groups	MIU Tech Circle	Feature
Professionals & graduates	General users	MIU CS students, alumni, faculty	Target Audience
Student-Centered Design	✗	✗	✓
Academic Year Consideration	✗	✗	✓
Safe & Official Environment	✗	✗	✓
Organized Opportunities	✗	✗	✓
Alumni as Mentors	✗	✗	✓
Event & Announcement Tracking	✗	✗	✓
Calendar & Notifications	✗	✗	✓
User Verification	✗	✗	✓

Our System Functionalities

- User Authentication
- Posts & Announcements Management
- Stories [24-Hour Content]
- Real-Time Chatting
- Freelancing Hub
- Notifications System

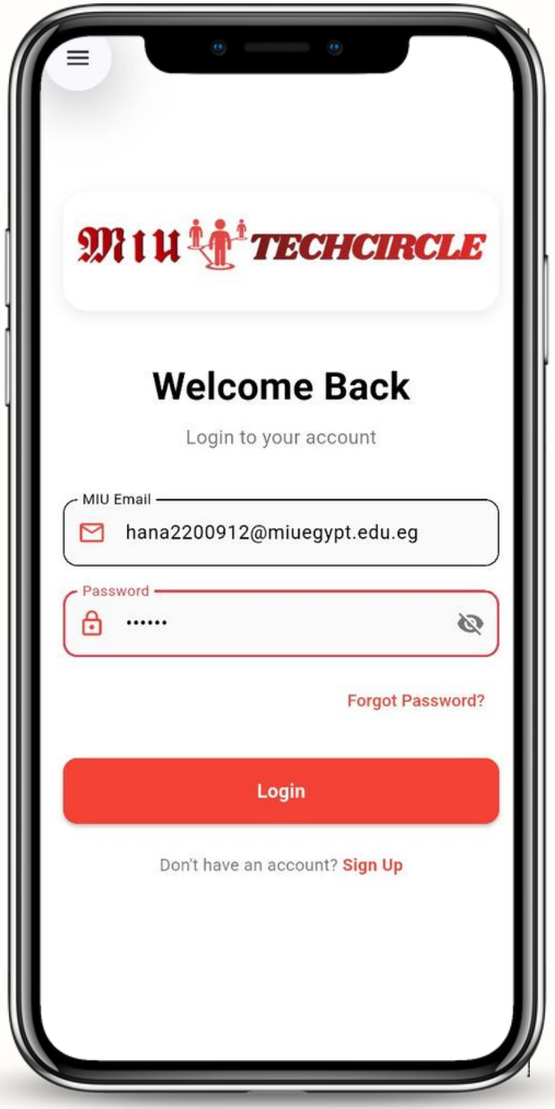


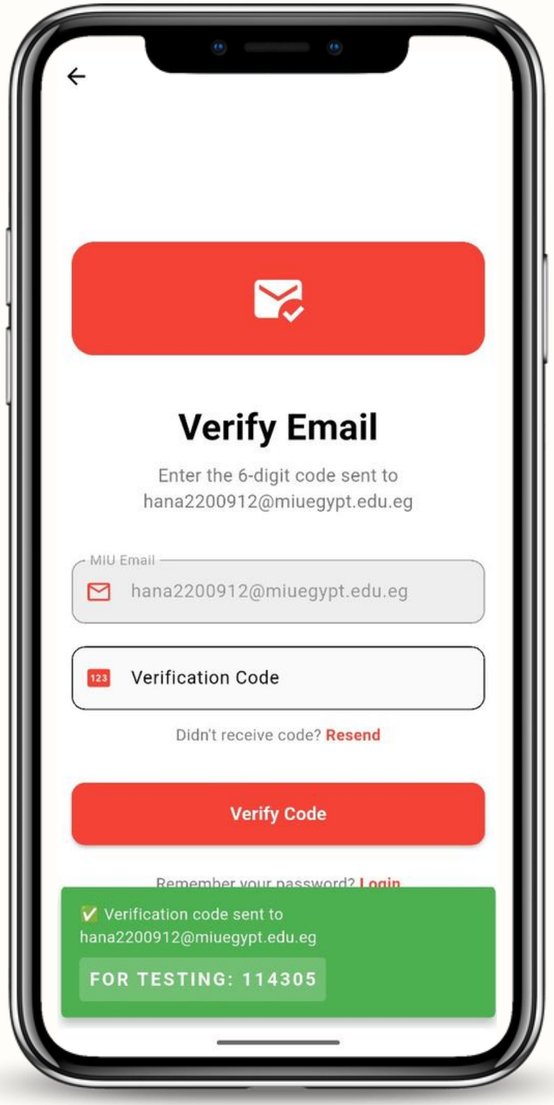


The Application supports both the light and the dark mode

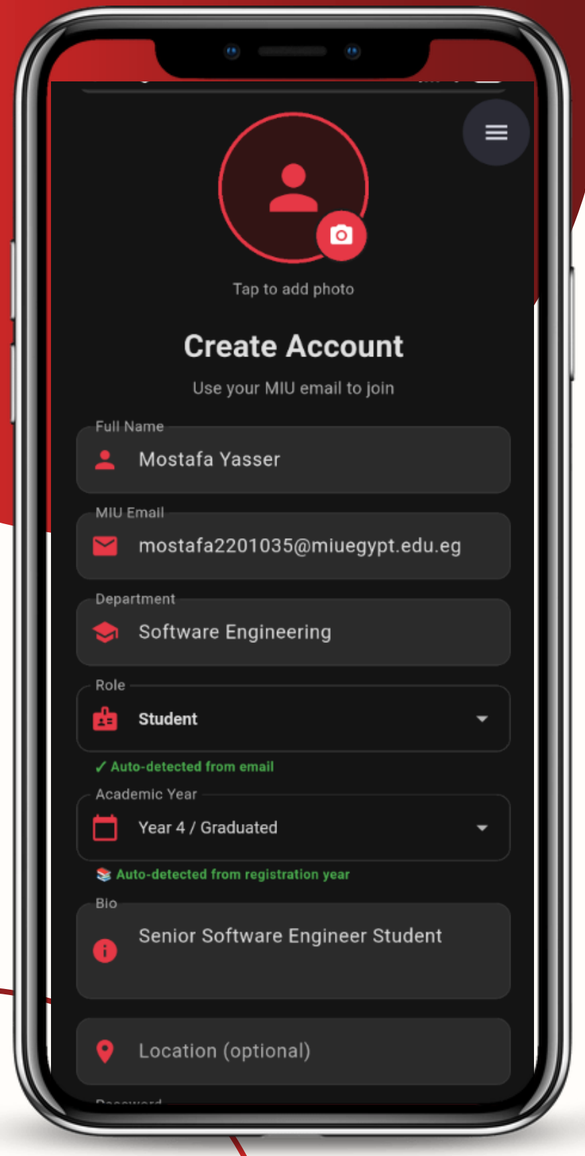
As well as it supports the rotation of the mobile

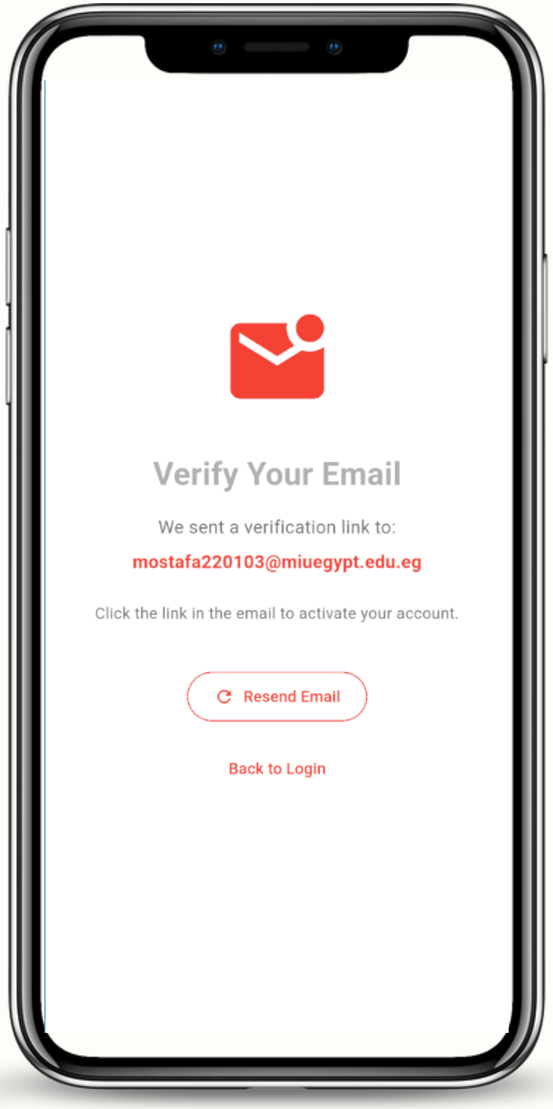
Login Page & Forget Password

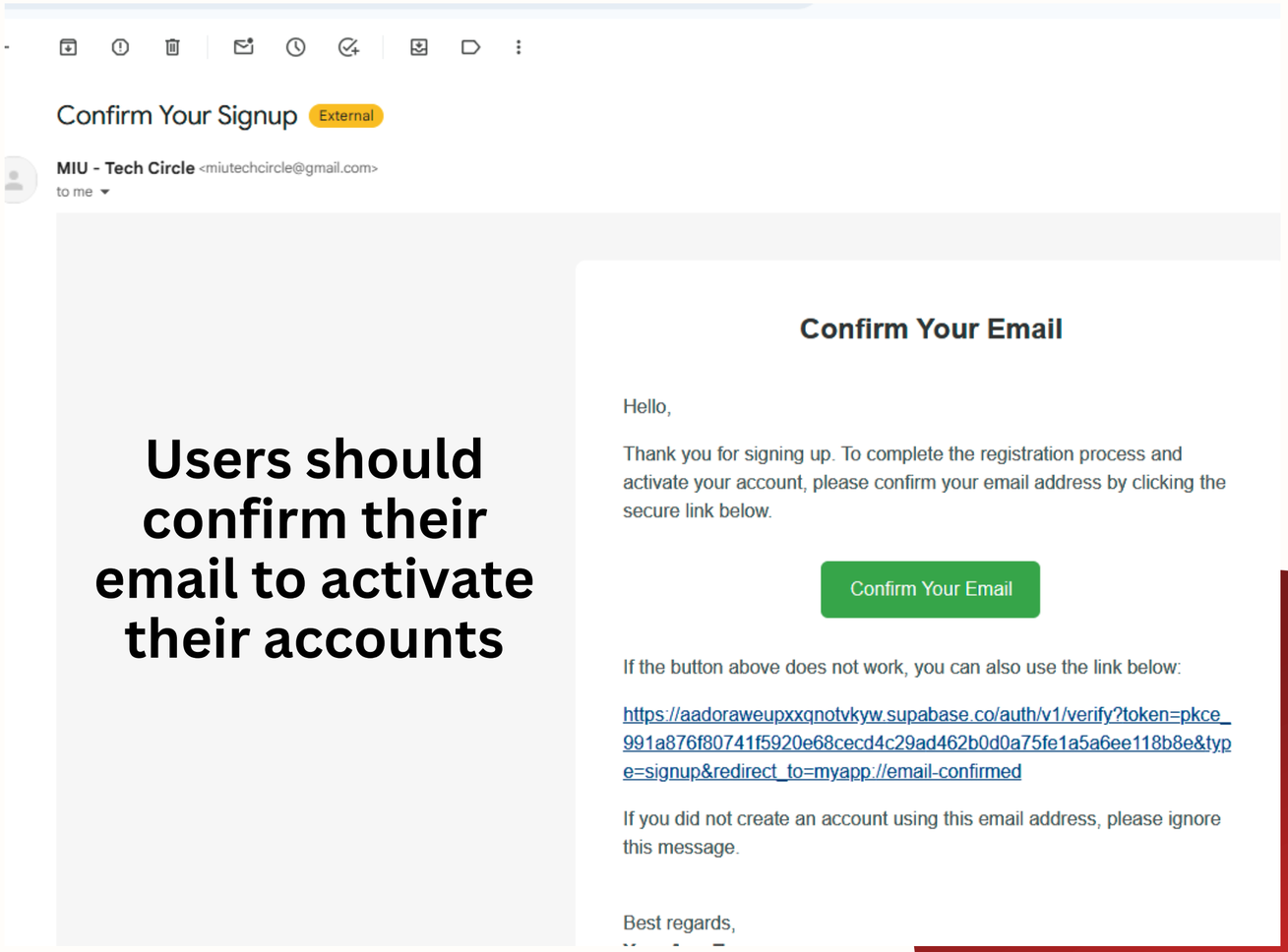




Sign Up Page

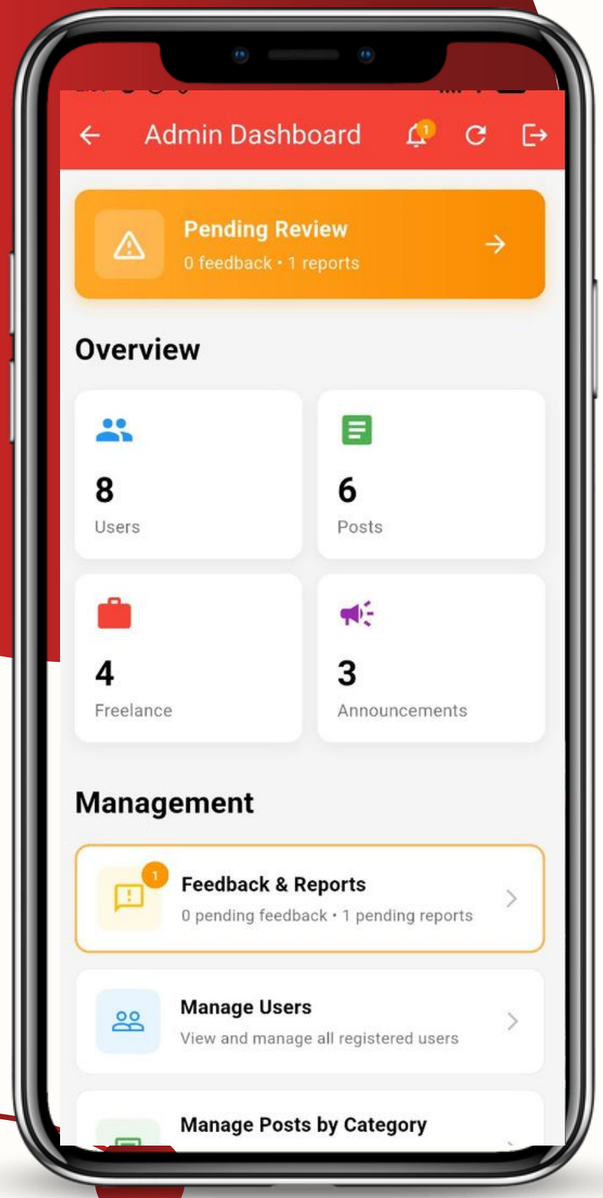


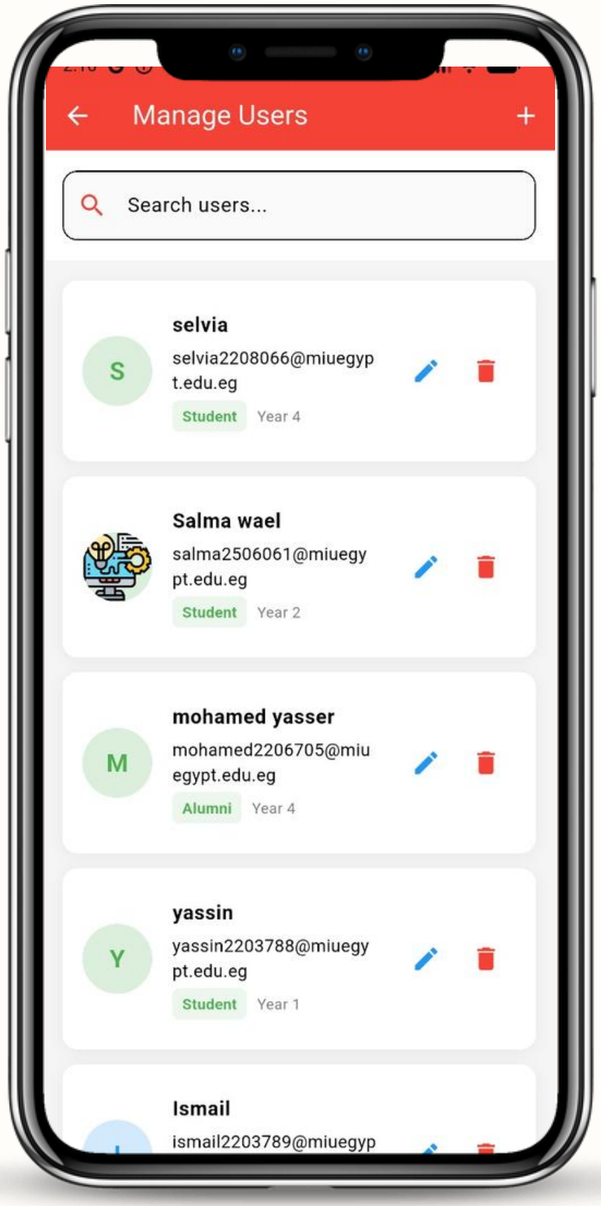


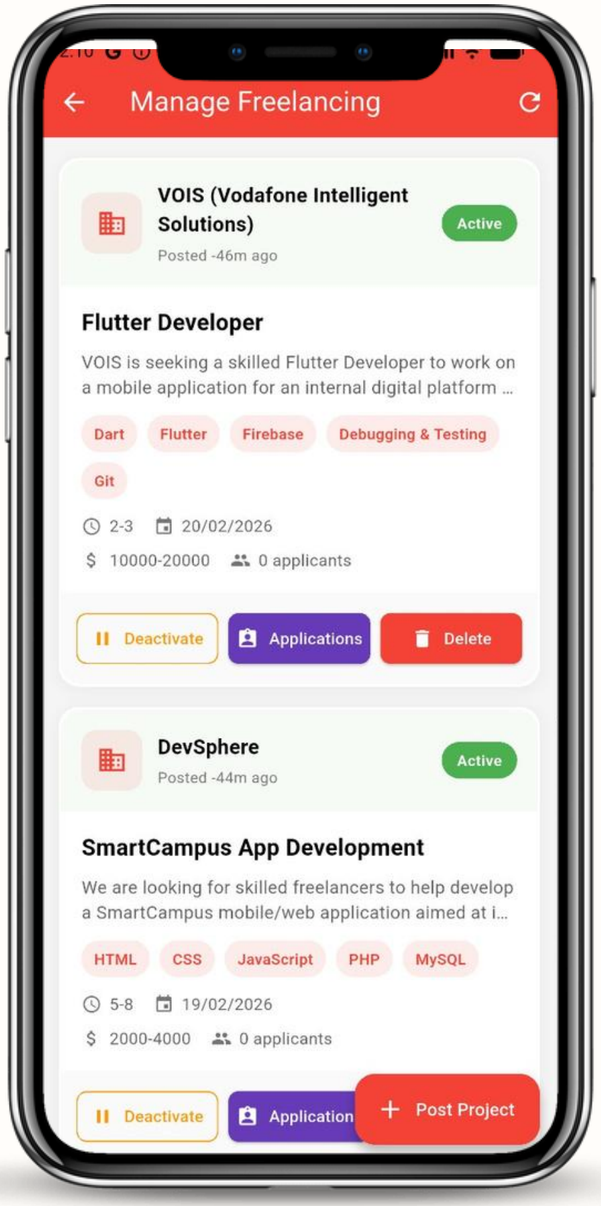


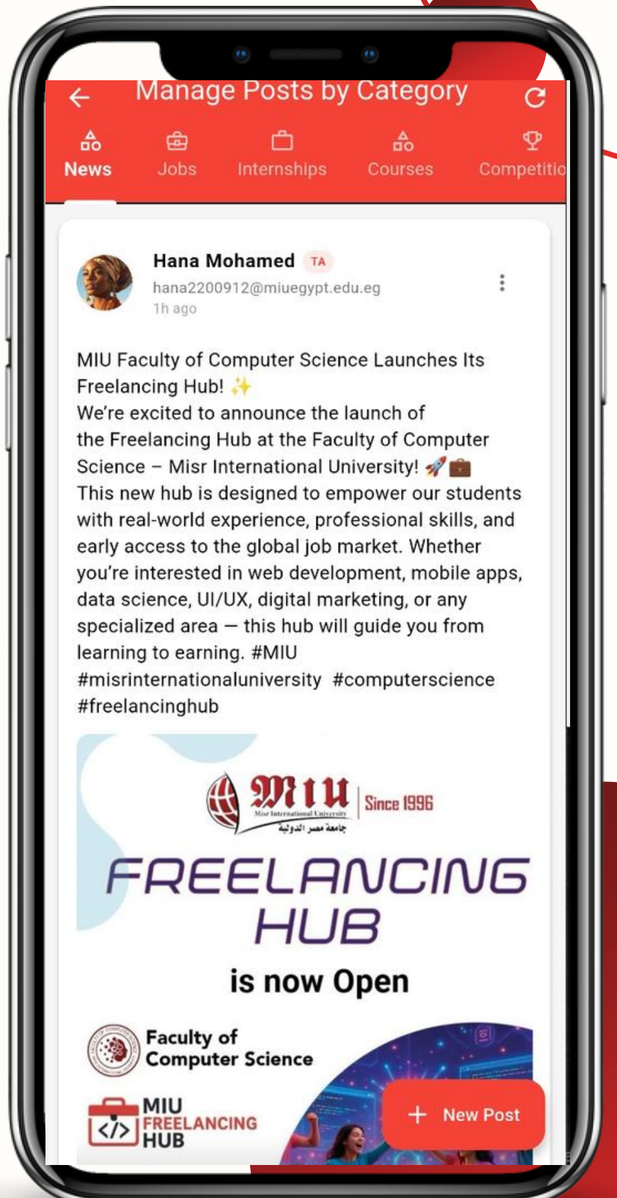
Users should confirm their email to activate their accounts

Admin Dashboard Page

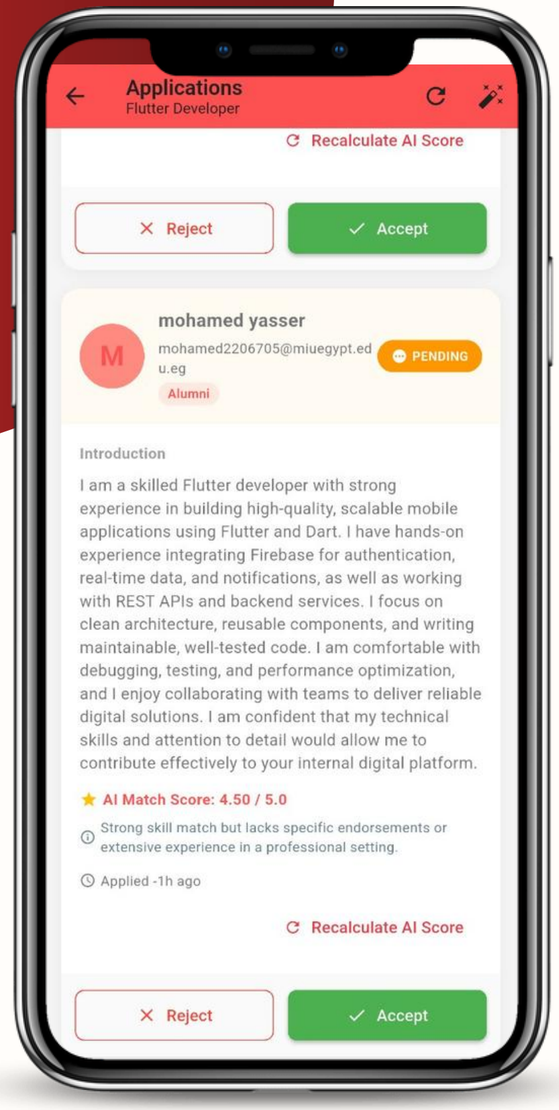








Match Score for project applications using AI



Uses AI to score how well applicants match job requirements.

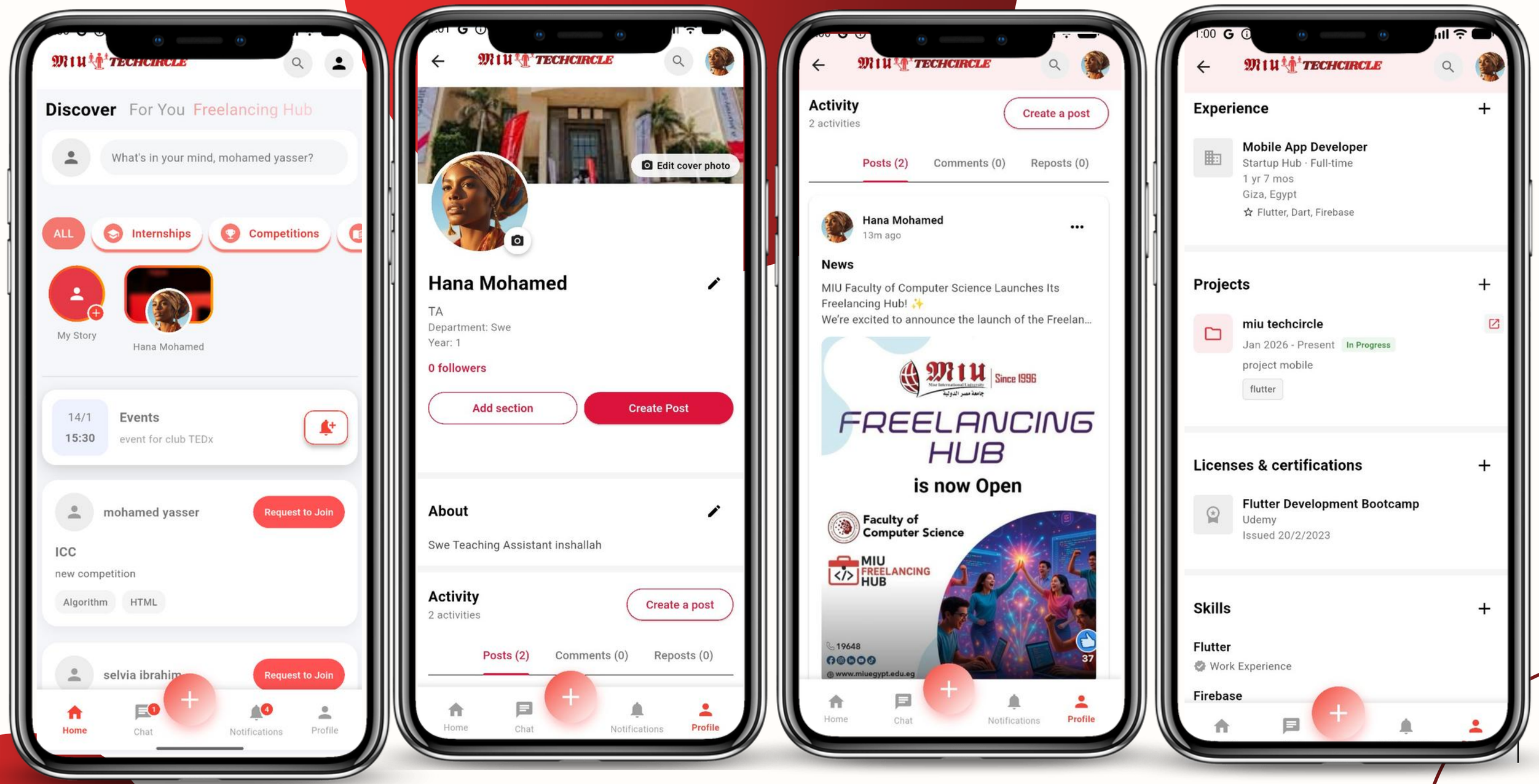
Functionalities:

- Sends applicant data + project requirements to OpenAI GPT-4
- Returns match score (0.0-5.0) with reasoning
- Single applicant recalculation via button click
- Batch analysis: processes 5 applications simultaneously with progress dialog
- Scores saved to database and displayed in real-time UI

```dart
class AIService {
  static String get _apiKey => dotenv.env['OPENAI_API_KEY'] ?? '';
  static const String _baseUrl = 'https://api.openai.com/v1/chat/completions';

  static Future<Map<String, dynamic>> analyzeApplication({
    required List<String> userSkills,
    required List<String> userExperiences,
    required List<String> userLicenses,
    required String introduction,
    required List<String> projectSkills,
    required String projectDescription,
    // New Profile Attributes
    String? userRole,
    String? userDepartment,
    String? userAcademicYear,
    String? userBio,
    String? userLocation,
  }) async {
    try {
      if (_apiKey.isEmpty || _apiKey == 'YOUR_OPENAI_API_KEY_HERE') {
        debugPrint('⚠️ OpenAI API Key is missing');
        return {'score': 0.0, 'reason': 'API Key missing'};
      }
      final prompt =
          '''
      You are an AI recruiter. Analyze the following job application against the project requirements.
      Project Description: "$projectDescription"
      Required Skills: ${projectSkills.join(', ')}

      Applicant Profile:
      - Role: ${userRole ?? 'N/A'}
      - Department: ${userDepartment ?? 'N/A'}
      - Academic Year: ${userAcademicYear ?? 'N/A'}
      - Location: ${userLocation ?? 'N/A'}
      - Bio: "${userBio ?? 'N/A'}"

      Applicant Skills: ${userSkills.join(', ')}
      Applicant Experience: ${userExperiences.join('; ')}
      Applicant Licenses/Certifications: ${userLicenses.join('; ')}
      Applicant Introduction: "$introduction"

      Task:
      1. Compare the applicant's profile, skills (names, proficiency, endorsements), experience, licenses, and introduction to the project requirements.
      2. Provide a match score from 0.0 to 5.0 using this rubric:
          - 5.0 (Perfect): Has ALL required skills at High/Expert level + relevant experience/endorsements/licenses.
          - 4.0-4.9 (Strong): Has all/most skills but lower proficiency, or missing only minor skills/experience.
          - 3.0-3.9 (Moderate): Missing one key skill or low proficiency in key areas, or limited experience.
          - < 3.0 (Weak): Missing multiple key skills or relevant experience.
      3. Provide a concise 1-sentence reason for the score.
      Return ONLY a JSON object in this format:
      {
        "score": 4.5,
        "reason": "Strong skill match but lacks mentioned experience in..."
      }
```

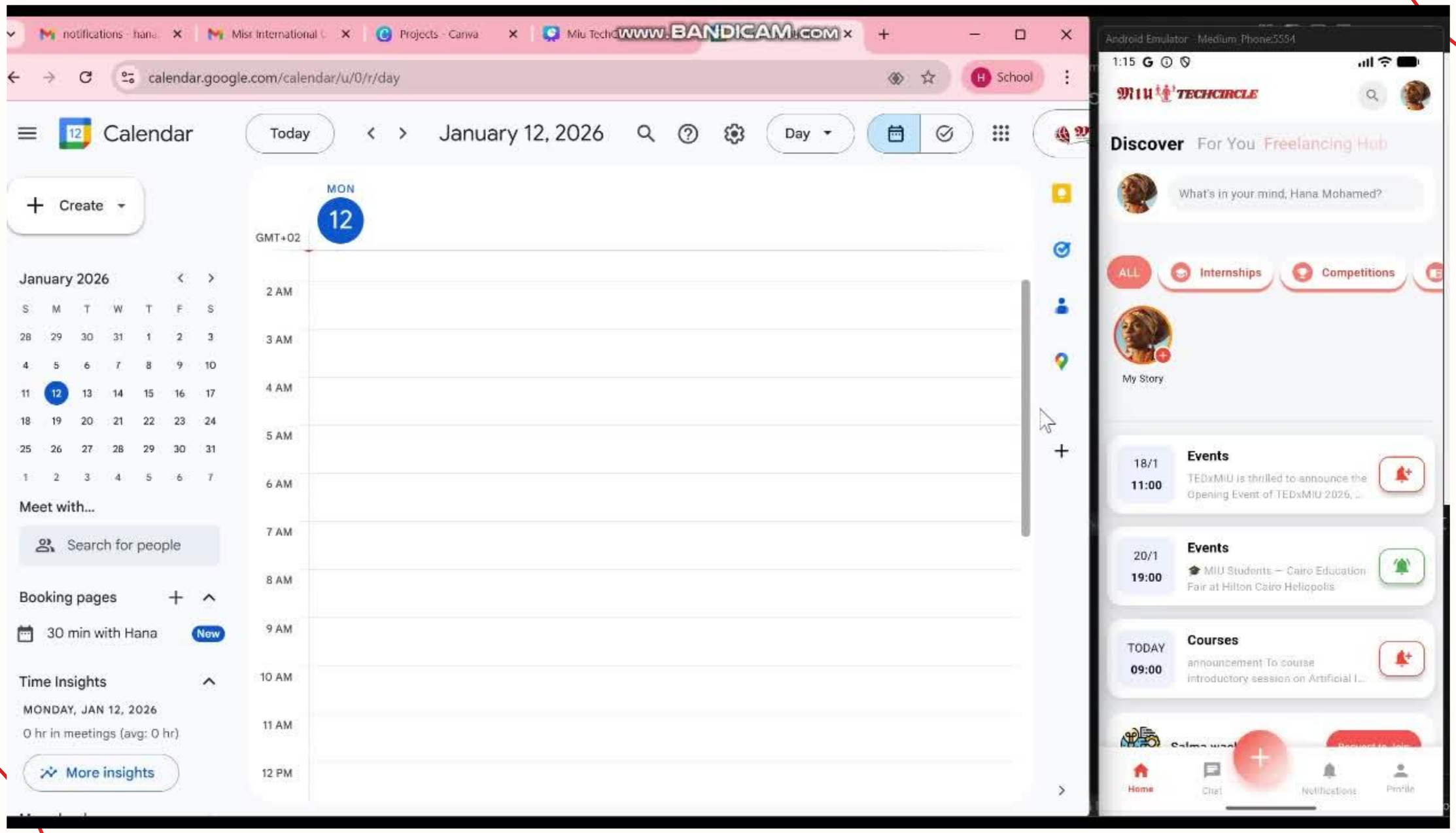
AiService Code:

Sends applicant data and project requirements to OpenAI GPT-4, which analyzes the match and returns a score (0.0-5.0) with reasoning.



Home Feed& Profile

Google Calendar API Demo



Code: Google Calendar API

Syncs app announcements/events to user's Google Calendar with add/remove functionality.

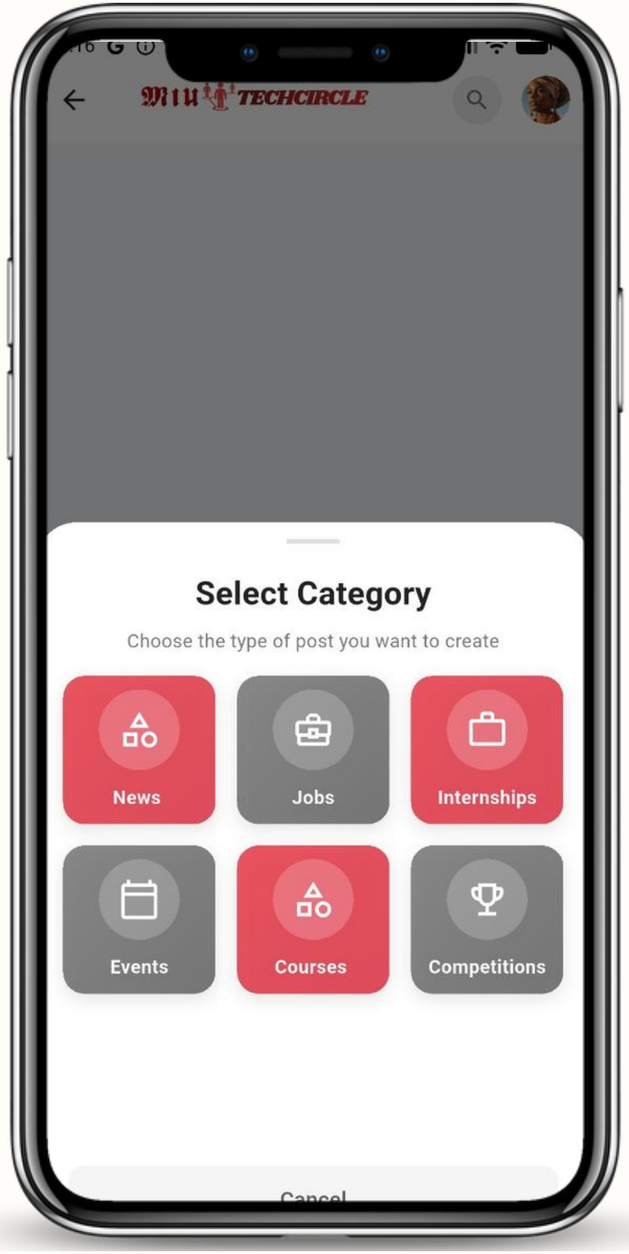
Authenticates user with Google and creates an API client for calendar operations.

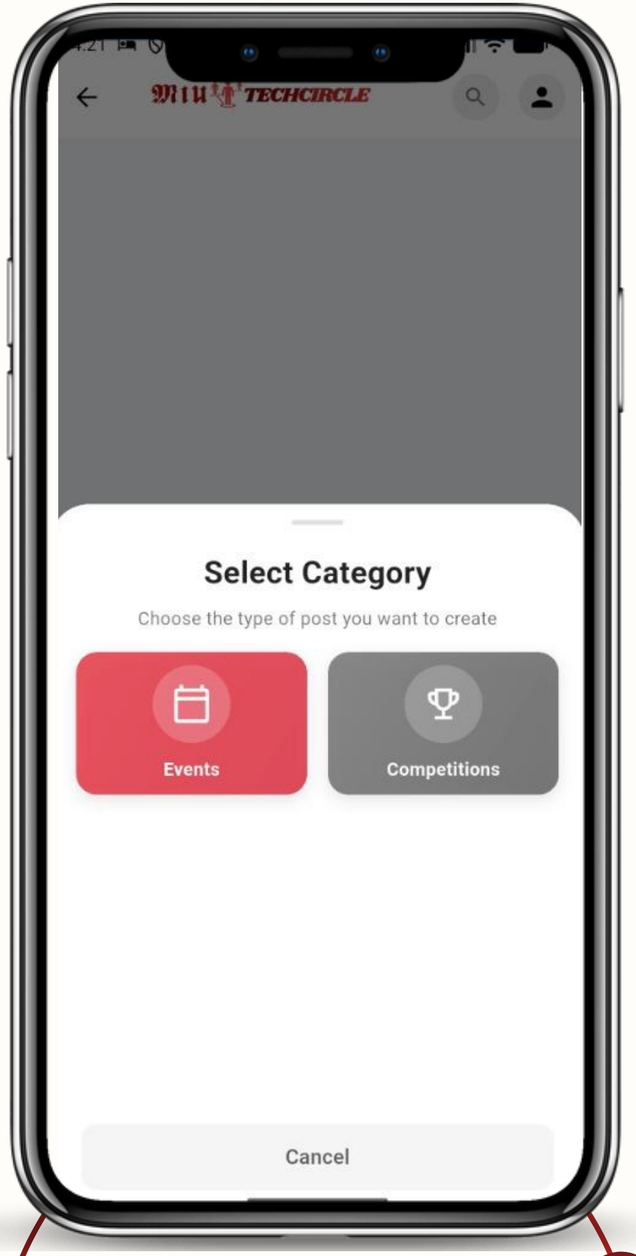
```
class GoogleCalendarService {
  static final GoogleSignIn _googleSignIn = GoogleSignIn(
    scopes: [
      'https://www.googleapis.com/auth/calendar.events',
    ],
  );

  // 🗑️ Delete event from Google Calendar
  static Future<void> deleteEvent(String eventId) async {
    final account = await _googleSignIn.signIn();
    if (account == null) return;

    final authHeaders = await account.authHeaders;
    final authClient = GoogleAuthClient(authHeaders);
    final calendarApi = gcal.CalendarApi(authClient);
```

Creates a calendar event from announcement data, adds it to user's Google Calendar, and returns the event ID to store in database.

```
//  Add event and RETURN the event ID
static Future<String?> addEvent({
  required String title,
  required String description,
  required DateTime startTime,
  required DateTime endTime,
}) async {
  final account = await _googleSignIn.signIn();
  if (account == null) {
    throw Exception("User cancelled Google Sign-In");
  }

  final authHeaders = await account.authHeaders;
  final authClient = GoogleAuthClient(authHeaders);

  final calendarApi = gcal.CalendarApi(authClient);

  final event = gcal.Event(
    summary: title,
    description: description,
    start: gcal.EventDateTime(
      dateTime: startTime.toUtc(),
    ), // gcal.EventDateTime
    end: gcal.EventDateTime(
      dateTime: endTime.toUtc(),
    ), // gcal.EventDateTime
  ); // gcal.Event

  // 🔑 Insert and return the event ID
  final createdEvent = await calendarApi.events.insert(event, "primary");
  return createdEvent.id; // This is what you need to store!
}
```

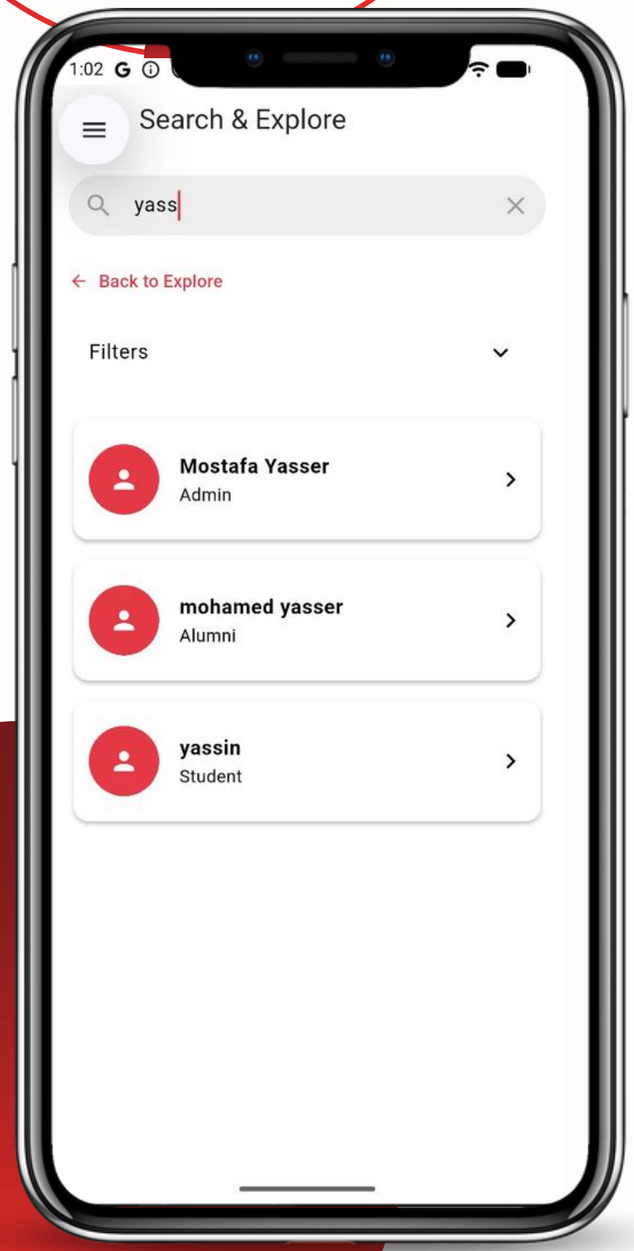
Select category to post “Instructor/ TA”





Select category to post “Student”

Search & Adding Stories



hanawahaaab@gmail.com's Projec | Integrations | Supabase | Claude

supabase.com/dashboard/project/aadoraweupxxqnotvkyw/integrations/cron/jobs

hana's Org FREE / hanawahaaab@gmail.com's Project / main PRODUCTION Connect Feedback Search... ⌘K

Integrations

EXPLORE

- All
- Wrappers
- Postgres Modules

INSTALLED

- Cron
- GraphQL
- Vault BETA

Integrations > Cron

Cron

Schedule recurring Jobs in Postgres

Overview Jobs

Search for a job

Refresh Create job

Name	Schedule	Last run	Next run	Command	Active
No name provided	* * * * *	12 Jan 2026 03:41:00 (+0200) Succeeded	12 Jan 2026 03:42:00 (+0200)	DELETE FROM stories WHERE created_at < (NOW	

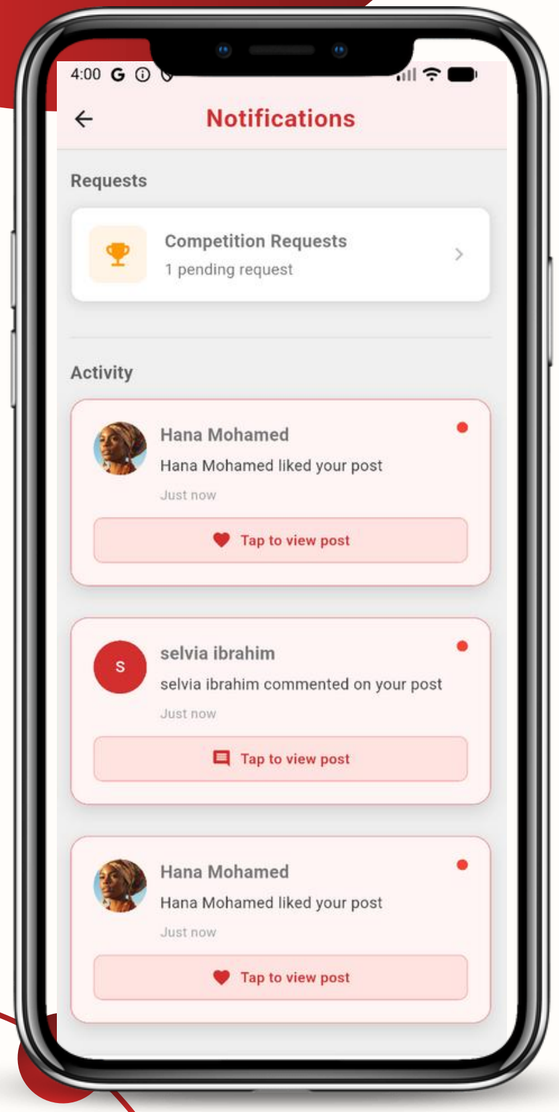
Command

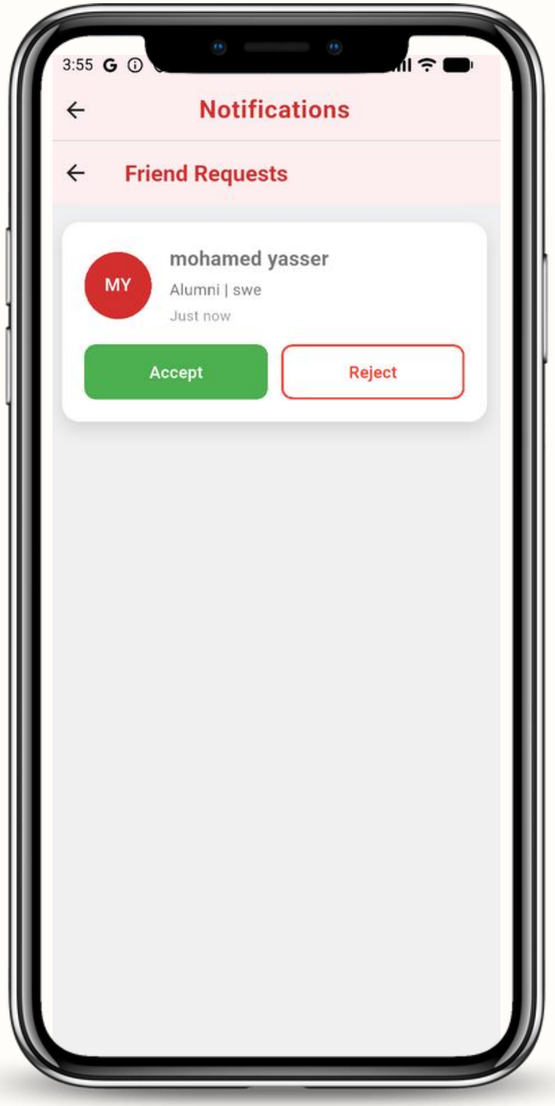
```
DELETE FROM stories
  WHERE created_at < (NOW() AT TIME ZONE 'UTC') -
INTERVAL '24 hours';
```

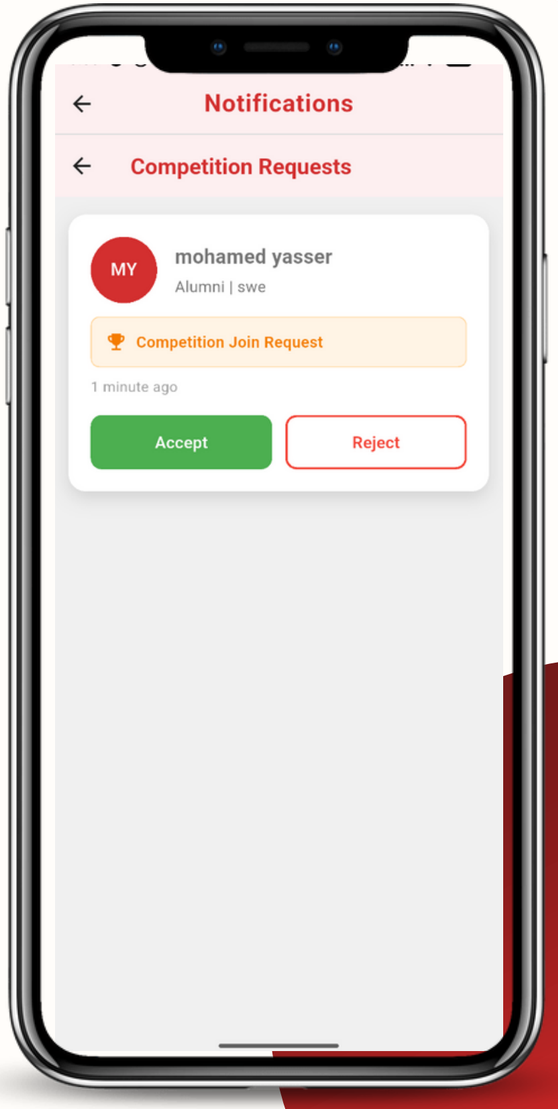
Total: 1 jobs

ENG 03:41 AM 12/01/2026

Notifications





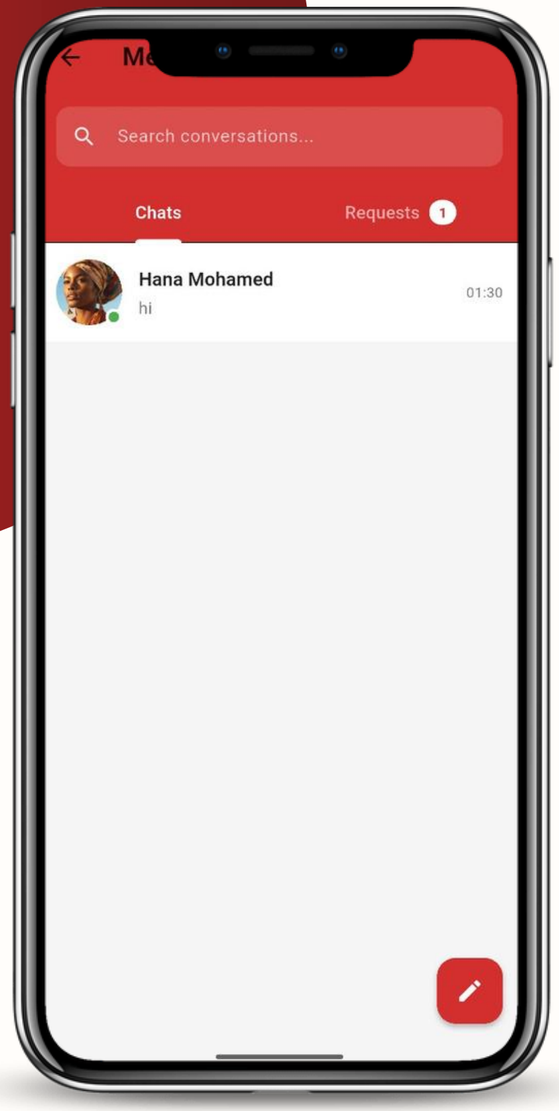


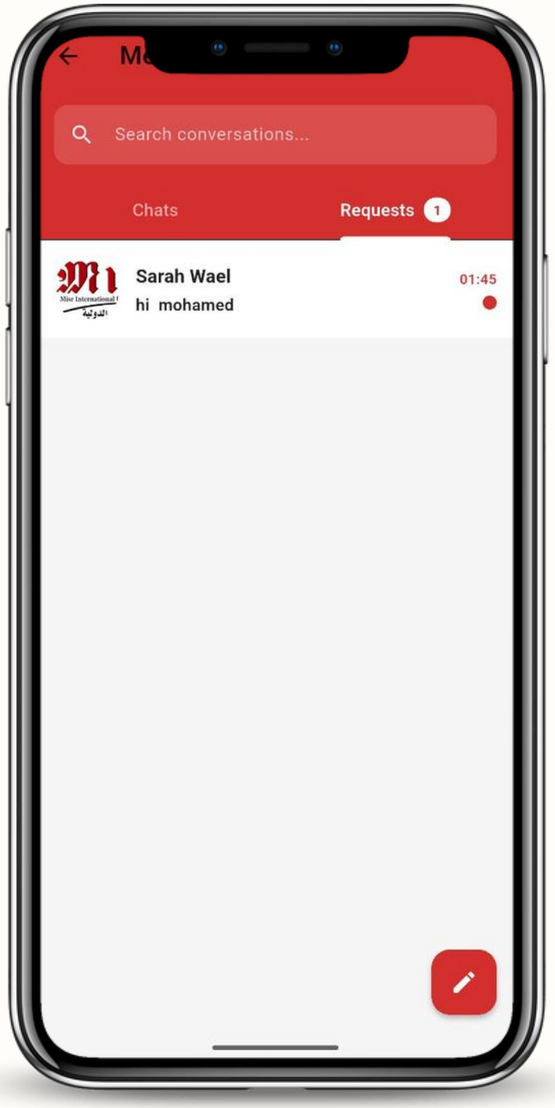
Notifications Provider

```dart
/// Fetch initial unread count
Future<void> fetchUnreadCount() async {
  if (_userId == null) {
    print('⚠️ Cannot fetch unread count: userId is null');
    return;
  }

  try {
    print('🔔 Fetching unread notification count for user: $_userId');

    // Count unread general notifications (excluding follow_request which are in folders)
    final generalNotifications = await supabase
        .from('notifications')
        .select('notification_id')
        .eq('user_id', _userId!)
        .eq('is_read', false)
        .neq('type', 'follow_request'); // Exclude follow requests

    // Count pending friend requests
    final friendRequests = await supabase
        .from('friendship_requests')
        .select('request_id')
        .eq('receiver_id', _userId!)
        .eq('status', 'pending');

    // Count pending competition requests
    final competitionRequests = await supabase
        .from('team_join_requests')
        .select('request_id')
        .eq('competition_owner_id', _userId!)
        .eq('status', 'pending');

    final totalCount = generalNotifications.length +
        friendRequests.length +
        competitionRequests.length;

    _unreadCount = totalCount;
    print('✅ Total unread notifications: $_unreadCount (General: ${generalNotifications.length}, Friends: ${friendRequests.length}, Competitions: ${competitionRequests.length})');
    notifyListeners();
  } catch (e) {
    print('❌ Error fetching unread count: $e');
  }
}
```

- Called when: App starts, user logs in, notification is marked as read, or request is accepted/rejected
- Queries 3 tables (notifications, friend requests, competition requests)
- Counts unread/pending items
- Returns single number for the notification badge

Messaging & Chats





Code: Messaging & Chats

sendMessage()

```
final messageText = messageController.text.trim();
messageController.clear();

setState(() => isSending = true);

try {
  await supabase.from('messages').insert({
    'conversation_id': conversationId,
    'sender_id': currentUserId,
    'content': messageText,
    'created_at': DateTime.now().toIso8601String(),
    'is_read': false,
  }).select().single();

 // Update conversation timestamp
  await supabase
      .from('conversations')
      .update({'updated_at': DateTime.now().toIso8601String()})
      .eq('id', conversationId!);
```

State Management using Riverpod Notifier:

ChatsNotifier manages a list of chats, auto-subscribes to database changes, and updates state when new messages arrive, triggering UI rebuilds.

```
// StateNotifier for managing chats list
class ChatsNotifier extends StateNotifier<List<Chat>> {
  final int currentUserId;
  RealtimeChannel? _messagesChannel;
  RealtimeChannel? _settingsChannel;

  ChatsNotifier(this.currentUserId) : super([]) {
    _setupRealtimeSubscriptions();
  }

 void _setupRealtimeSubscriptions() {…
  @override
  void dispose() {…
```

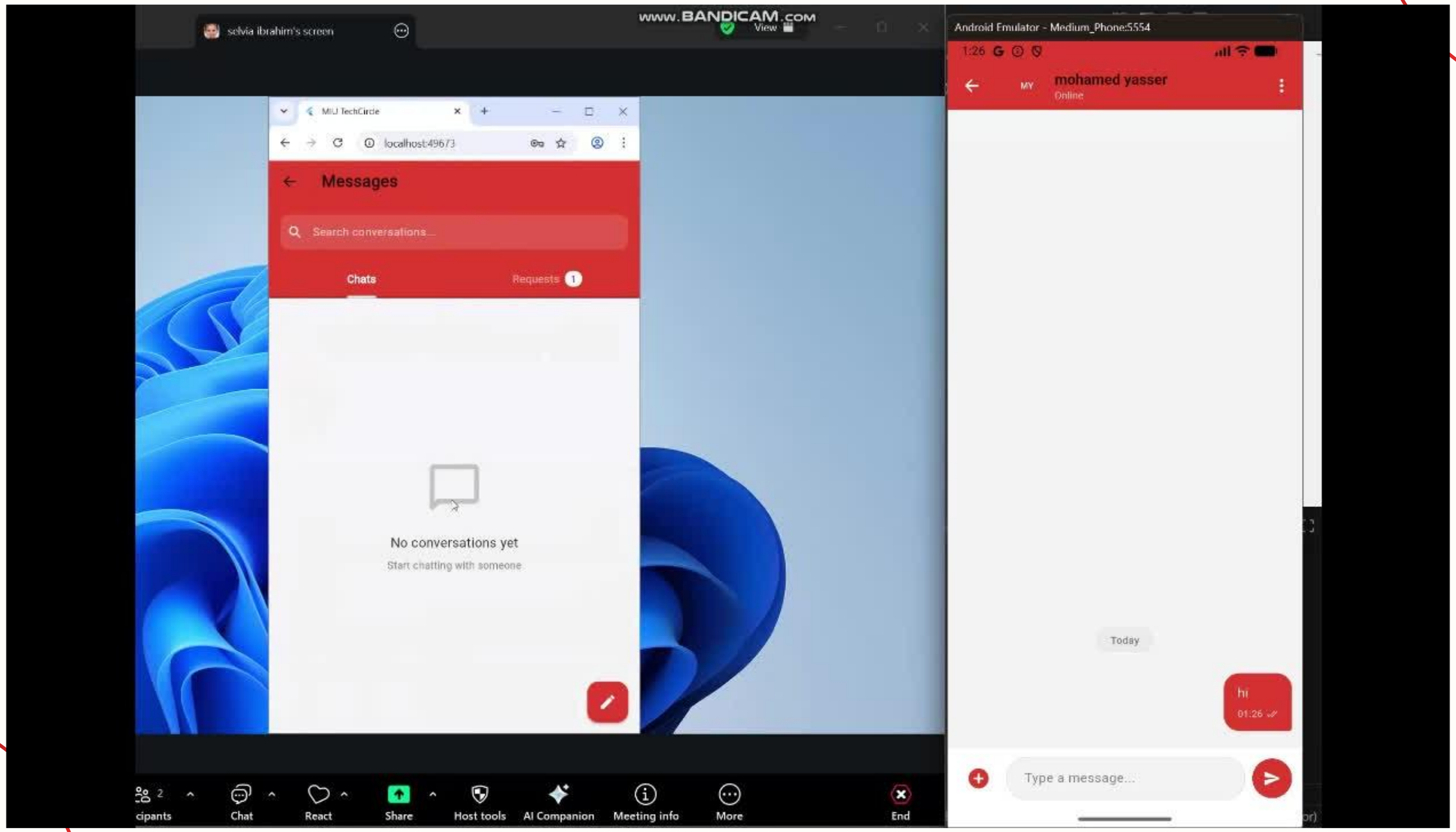
Messages Provider Code:

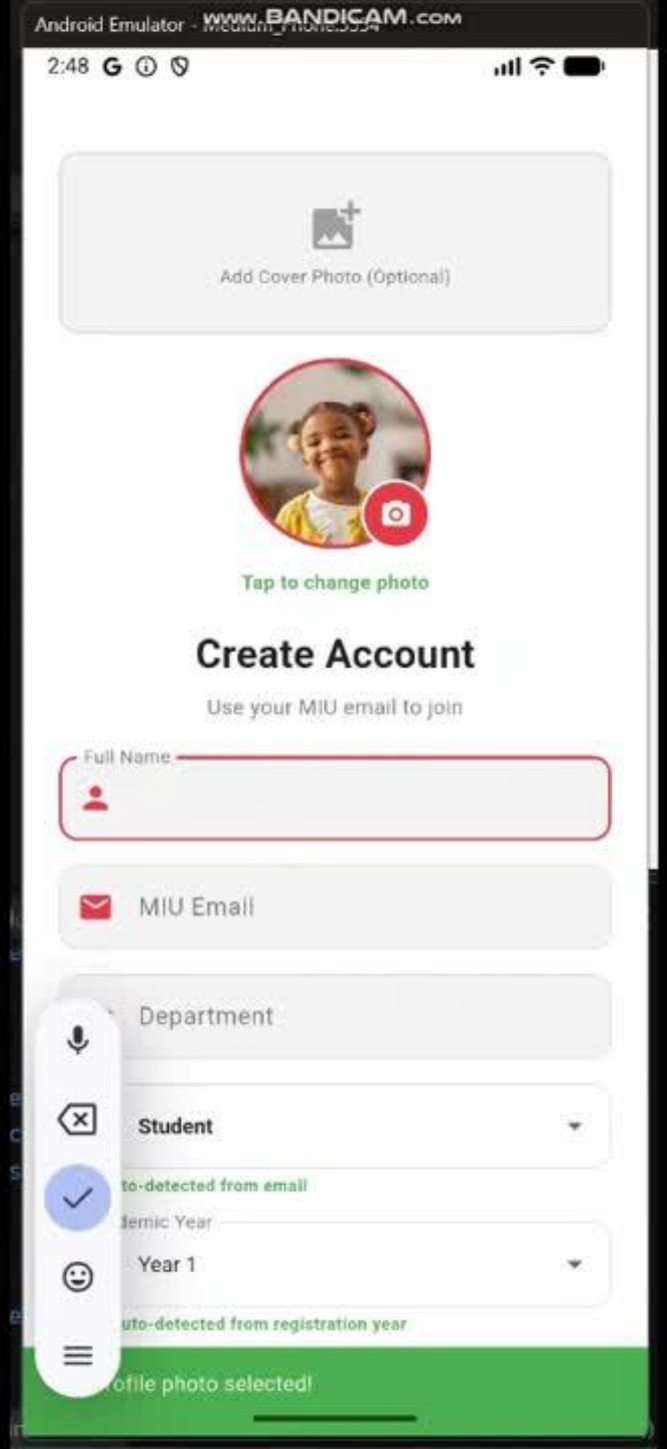
```
Future<void> loadUnreadCount(int currentUserId) async {
  _isLoading = true;
  notifyListeners();

  try {
    debugPrint('📊 Loading unread messages for user $currentUserId...');

    final myConversations = await _supabase
        .from('conversation_participants')
        .select('conversation_id')
        .eq('user_id', currentUserId);

    if ((myConversations as List).isEmpty) {
      _unreadCount = 0;
      _unreadSenders.clear();
      debugPrint('✅ No conversations found');
      _isLoading = false;
      notifyListeners();
      return;
    }

    final conversationIds = myConversations
        .map((p) => p['conversation_id'] as String)
        .toList();

    debugPrint('🔍 Found ${conversationIds.length} conversations');
```

- We track unread messages using a MessageProvider.
- loadUnreadCount() fetches messages from Supabase, filtering only accepted conversations.
- Unread messages are counted per sender and stored in _unreadSenders.
- The Bottom Navigation Bar listens to the provider using Consumer<MessageProvider>.
- badgeCount on the chat icon updates dynamically, reflecting real-time unread messages.
- Marking messages as read updates _readSenders and refreshes the badge immediately.

```
Expanded(
  child: Consumer<MessageProvider>(
    builder: (context, messageProvider, child) {
      // ✅ Get the actual unread count from provider
      final unreadCount = messageProvider.unreadCount;

      return _NavItemWithBadge(
        icon: Icons.chat,
        label: "Chat",
        selected: currentIndex == 1,
        badgeCount: unreadCount, // ✅ Use dynamic count
        onTap: () => _onItemTapped(context, 1),
      ); // _NavItemWithBadge
    },
  ), // Consumer

), // Expanded
```

Messaging & Chats Demo




Android Emulator
2:48
Add Cover Photo (Optional)
Tap to change photo
Create Account
Use your MIU email to join
Full Name
MIU Email
Department
Student
Year 1
ofile photo selected!

Full Demo

Thank You